

C++ Programming

Day 12: STL 활용 심화

priority_queue, Lambda Capture, find_if, count_if, transform, Object Container

Contents

1	학습 목표	2
2	Day 12 개요	2
3	priority_queue	2
3.1	queue와의 차이	2
3.2	주요 함수	3
3.3	최종 코드	3
3.4	실습	4
4	Lambda Capture	4
4.1	값 캡처	4
4.2	참조 캡처	4
4.3	최종 코드	5
4.4	실습	5
5	find_if	6
5.1	기본 구조	6
5.2	find와 find_if 비교	6
5.3	최종 코드	7
5.4	실습	7
6	count_if	8
6.1	Predicate	8
6.2	최종 코드	8
6.3	실습	9
7	transform	9
7.1	기본 구조	9
7.2	출력 공간	9
7.3	최종 코드	10
7.4	실습	11
8	Object Container	11
8.1	vector에 객체 저장	11
8.2	객체와 find_if	12
8.3	최종 코드	12
8.4	실습	13
9	개념 연결	15
10	종합 정리	15
11	실습 파일 구성	16

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- `priority_queue`의 우선순위 기반 처리 방식 이해하기
- `top()`, `push()`, `pop()`을 이용해 우선순위 큐 사용하기
- 람다 캡처에서 값 캡처와 참조 캡처의 차이 이해하기
- `find_if()`와 람다를 이용해 조건을 만족하는 첫 원소 찾기
- `count_if()`와 predicate를 이용해 조건을 만족하는 원소 개수 세기
- `transform()`을 이용해 범위의 원소를 새로운 값으로 변환하기
- 사용자 정의 클래스 객체를 `vector`에 저장하기
- 클래스, 컨테이너, 반복자, 람다, STL 알고리즘을 하나의 코드에서 결합하기

2. Day 12 개요

Day 10에서는 `vector`, 반복자, 기본 STL 알고리즘과 람다 표현식을 학습했고, Day 11에서는 `set`, `map`, `unordered_map`, `stack`, `queue`를 학습했다. Day 12에서는 이 내용을 확장하여 우선순위 기반 컨테이너와 조건 기반 STL 알고리즘을 다루고, 마지막에는 클래스 객체를 STL 컨테이너에 저장하여 여러 개념을 연결한다.

주제	핵심 역할
<code>priority_queue</code>	우선순위가 높은 원소부터 처리
Lambda Capture	람다 바깥 변수 사용
<code>find_if()</code>	조건을 만족하는 첫 원소 탐색
<code>count_if()</code>	조건을 만족하는 원소 개수 계산
<code>transform()</code>	범위의 원소를 규칙에 따라 변환
Object Container	클래스와 STL 알고리즘 결합

핵심 포인트

Day 12의 핵심은 STL 함수를 개별적으로 외우는 것이 아니라 람다로 조건이나 변환 규칙을 만들고, 그 규칙을 STL 알고리즘에 전달하는 흐름을 이해하는 것이다.

3. `priority_queue`

`priority_queue`는 원소가 들어온 순서가 아니라 우선순위에 따라 원소를 처리하는 컨테이너 어댑터이다. 기본 `priority_queue<int>`에서는 가장 큰 값이 가장 높은 우선순위를 가진다.

3.1. `queue`와의 차이

일반적인 `queue`는 FIFO 구조이므로 먼저 들어온 원소가 먼저 나온다. 반면 `priority_queue`는 현재 저장된 원소 중 우선순위가 가장 높은 원소를 `top()`으로 제공한다.

컨테이너	처리 기준	접근 함수
queue	먼저 들어온 원소	front()
priority_queue	우선순위가 높은 원소	top()

3.2. 주요 함수

함수	역할
push(value)	원소 추가
top()	가장 높은 우선순위의 원소 접근
pop()	가장 높은 우선순위의 원소 제거
size()	원소 개수 반환
empty()	비어 있는지 확인

priority_queue는 모든 원소를 완전히 정렬된 배열처럼 유지하는 컨테이너가 아니다. 내부적으로 우선순위가 가장 높은 원소를 빠르게 얻을 수 있는 형태로 관리한다. 기본 구현에서는 힙 자료구조가 사용된다.

3.3. 최종 코드

```

1 #include <iostream>
2 #include <queue>
3
4 using namespace std;
5
6 int main()
7 {
8     priority_queue<int> pq;
9
10    pq.push(30);
11    pq.push(10);
12    pq.push(50);
13    pq.push(20);
14    pq.push(40);
15
16    cout << "Size: " << pq.size() << endl;
17    cout << "Top: " << pq.top() << endl;
18
19    while (!pq.empty())
20    {
21        cout << pq.top() << " ";
22        pq.pop();
23    }
24
25    cout << endl;
26
27    return 0;
28 }
```

출력 결과는 다음과 같다.

```

Size: 5
Top: 50
50 40 30 20 10
```

3.4. 실습

실습

정수 점수 75, 92, 60, 88, 100을 `priority_queue<int>`에 저장한다. 가장 높은 점수를 먼저 출력하고, 이후 모든 점수를 우선순위가 높은 순서대로 출력한다.

```

1 #include <iostream>
2 #include <queue>
3
4 using namespace std;
5
6 int main()
7 {
8     priority_queue<int> pq;
9
10    pq.push(75);
11    pq.push(92);
12    pq.push(60);
13    pq.push(88);
14    pq.push(100);
15
16    cout << "Highest score: " << pq.top() << endl;
17
18    while (!pq.empty())
19    {
20        cout << pq.top() << " ";
21        pq.pop();
22    }
23
24    cout << endl;
25
26    return 0;
27 }
```

4. Lambda Capture

Day 10에서 람다 표현식의 기본 문법을 학습했다. 람다 캡처(lambda capture)는 람다 바깥에서 선언된 지역 변수를 람다 내부에서 사용할 수 있게 하는 기능이다.

4.1. 값 캡처

대괄호 안에 변수 이름을 작성하면 해당 변수의 값을 캡처한다.

```

1 int number = 10;
2
3 auto show = [number]()
4 {
5     cout << number << endl;
6 };
```

[number]는 람다가 만들어지는 시점의 `number` 값을 람다 내부에 복사한다. 따라서 바깥 변수가 이후 변경되더라도 이미 캡처된 값은 별도로 유지된다.

4.2. 참조 캡처

변수 이름 앞에 `&`를 붙이면 참조로 캡처한다.

```

1 int number = 10;
2
3 auto update = [&number]()
4 {
5     number += 5;
6 };

```

이 경우 람다 내부의 `number`는 바깥 변수 자체를 참조하므로 람다 내부에서 변경하면 원본 값도 변경된다.

캡처	의미	원본 변경
<code>[number]</code>	값 복사	직접 변경하지 않음
<code>[&number]</code>	참조	변경 가능

4.3. 최종 코드

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int number = 10;
8
9     auto copyLambda = [number]()
10    {
11        cout << "Copy: " << number << endl;
12    };
13
14    auto refLambda = [&number]()
15    {
16        number += 5;
17        cout << "Reference: " << number << endl;
18    };
19
20    copyLambda();
21    refLambda();
22
23    cout << "Outside: " << number << endl;
24
25    return 0;
26 }

```

출력 결과는 다음과 같다.

```

Copy: 10
Reference: 15
Outside: 15

```

4.4. 실습

실습

`score = 80`을 선언한다. 값 캡처를 사용하는 람다에서 현재 점수를 출력하고, 참조 캡처를 사용하는 람다에서 점수에 10을 더한다. 두 람다 실행 후 바깥의 `score`를 출력한다.

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int score = 80;
8
9     auto printScore = [score]()
10    {
11        cout << "Score: " << score << endl;
12    };
13
14    auto plusTenScore = [&score]()
15    {
16        score += 10;
17        cout << "Updated score: " << score << endl;
18    };
19
20    printScore();
21    plusTenScore();
22
23    cout << "Outside: " << score << endl;
24
25    return 0;
26 }

```

5. find_if

`find_if()`는 지정한 범위에서 조건을 만족하는 첫 번째 원소를 찾는 STL 알고리즘이다. 정확한 값을 찾는 `find()`와 달리 세 번째 인자로 조건을 검사하는 함수를 전달한다.

5.1. 기본 구조

```
1 find_if(begin, end, condition);
```

람다를 조건으로 전달하면 각 원소에 대해 조건을 검사한다.

```

1 auto it = find_if(
2     numbers.begin(),
3     numbers.end(),
4     [](int n)
5     {
6         return n > 30;
7     }
8 );

```

조건을 만족하는 원소를 찾으면 해당 위치의 반복자를 반환한다. 끝까지 찾지 못하면 `end()`를 반환한다.

5.2. find와 find_if 비교

알고리즘	탐색 기준	반환값
<code>find()</code>	특정 값	반복자
<code>find_if()</code>	사용자 정의 조건	반복자

5.3. 최종 코드

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main()
8 {
9     vector<int> numbers = {10, 25, 30, 45, 50};
10
11     auto it = find_if(
12         numbers.begin(),
13         numbers.end(),
14         [](int n)
15         {
16             return n > 30;
17         }
18     );
19
20     if (it != numbers.end())
21     {
22         cout << "Found: " << *it << endl;
23     }
24     else
25     {
26         cout << "Not found" << endl;
27     }
28
29     return 0;
30 }
```

5.4. 실습

실습

3, 7, 12, 5, 18, 9가 저장된 벡터에서 `find_if()`와 람다를 사용하여 처음 등장하는 짝수를 찾는다. 짝수가 없으면 No even number를 출력한다.

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main()
8 {
9     vector<int> numbers = {3, 7, 12, 5, 18, 9};
10
11     auto it = find_if(
12         numbers.begin(),
13         numbers.end(),
14         [](int n)
15         {
16             return n % 2 == 0;
17         }
18     );
```



```

19
20     if (it != numbers.end())
21     {
22         cout << "First even number: " << *it << endl;
23     }
24     else
25     {
26         cout << "No even number" << endl;
27     }
28
29     return 0;
30 }

```

6. count_if

`count_if()`는 지정한 범위의 모든 원소를 검사하여 조건을 만족하는 원소의 개수를 반환한다. `find_if()`가 첫 번째 일치 위치를 찾는다면, `count_if()`는 전체 범위를 검사하여 개수를 계산한다.

6.1. Predicate

`find_if()`와 `count_if()`에 전달되는 조건 함수처럼 원소를 검사하여 `true` 또는 `false`를 반환하는 함수를 predicate라고 한다.

```

1 [](int n)
2 {
3     return n >= 20;
4 }

```

6.2. 최종 코드

```

1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main()
8 {
9     vector<int> numbers = {10, 25, 7, 30, 18, 40};
10
11     int count = count_if(
12         numbers.begin(),
13         numbers.end(),
14         [](int n)
15         {
16             return n >= 20;
17         }
18     );
19
20     cout << "Count: " << count << endl;
21
22     return 0;
23 }

```

출력 결과는 다음과 같다.

Count: 3

6.3. 실습

실습

점수 95, 72, 88, 61, 100, 79, 84가 저장된 벡터에서 `count_if()`와 람다를 사용하여 80 점 이상인 점수의 개수를 계산한다.

```

1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main()
8 {
9     vector<int> scores = {95, 72, 88, 61, 100, 79, 84};
10
11     int count = count_if(
12         scores.begin(),
13         scores.end(),
14         [](int n)
15         {
16             return n >= 80;
17         }
18     );
19
20     cout << "Scores above 80: " << count << endl;
21
22     return 0;
23 }
```

7. transform

`transform()`은 입력 범위의 각 원소에 변환 규칙을 적용하고 그 결과를 출력 범위에 저장하는 STL 알고리즘이다.

7.1. 기본 구조

```
1 transform(begin, end, output, operation);
```

첫 번째와 두 번째 인자는 읽을 입력 범위를 지정하고, 세 번째 인자는 변환 결과를 저장하기 시작할 위치를 지정한다. 네 번째 인자는 각 원소를 어떻게 변환할지 결정하는 함수이다.

```

1 transform(
2     numbers.begin(),
3     numbers.end(),
4     doubled.begin(),
5     [](int n)
6     {
7         return n * 2;
8     }
9 );
```

7.2. 출력 공간

다른 벡터에 결과를 저장하면서 `doubled.begin()`을 출력 위치로 사용할 경우, 결과를 저장할 공간이 미리 존재해야 한다.

```
1 vector<int> doubled(numbers.size());
```

원본 컨테이너 자체를 변환하려면 입력 범위와 출력 시작 위치에 같은 컨테이너를 사용할 수 있다.

```
1 transform(  
2     numbers.begin(),  
3     numbers.end(),  
4     numbers.begin(),  
5     [](int n)  
6     {  
7         return n * 2;  
8     }  
9 );
```

7.3. 최종 코드

```
1 #include <algorithm>  
2 #include <iostream>  
3 #include <vector>  
4  
5 using namespace std;  
6  
7 int main()  
8 {  
9     vector<int> numbers = {1, 2, 3, 4, 5};  
10    vector<int> doubled(numbers.size());  
11  
12    transform(  
13        numbers.begin(),  
14        numbers.end(),  
15        doubled.begin(),  
16        [](int n)  
17        {  
18            return n * 2;  
19        }  
20    );  
21  
22    cout << "Original: ";  
23  
24    for (int n : numbers)  
25    {  
26        cout << n << " ";  
27    }  
28  
29    cout << endl;  
30    cout << "Doubled: ";  
31  
32    for (int n : doubled)  
33    {  
34        cout << n << " ";  
35    }  
36  
37    cout << endl;  
38  
39    return 0;  
40 }
```

7.4. 실습

실습

점수 60, 70, 80, 90, 100을 `scores`에 저장한다. 원본은 변경하지 않고 `transform()`과 람다를 사용하여 각 점수에 5점을 더한 결과를 `bonusScores`에 저장한다.

```

1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main()
8 {
9     vector<int> scores = {60, 70, 80, 90, 100};
10    vector<int> bonusScores(scores.size());
11
12    transform(
13        scores.begin(),
14        scores.end(),
15        bonusScores.begin(),
16        [](int n)
17        {
18            return n + 5;
19        }
20    );
21
22    cout << "Original: ";
23
24    for (const auto& score : scores)
25    {
26        cout << score << " ";
27    }
28
29    cout << endl;
30    cout << "Bonus: ";
31
32    for (const auto& score : bonusScores)
33    {
34        cout << score << " ";
35    }
36
37    cout << endl;
38
39    return 0;
40 }
```

8. Object Container

지금까지는 주로 정수와 문자열을 STL 컨테이너에 저장했다. 클래스 객체 역시 `vector`와 같은 컨테이너에 저장할 수 있다. 이를 통해 클래스, 템플릿 기반 컨테이너, 반복자, 람다, 알고리즘을 하나의 흐름으로 연결할 수 있다.

8.1. vector에 객체 저장

다음과 같은 `Student` 클래스가 있다고 하자.

```

1 class Student
2 {
3 public:
4     string name;
5     int score;
6
7     Student(string name, int score)
8     {
9         this->name = name;
10        this->score = score;
11    }
12 };

```

객체의 자료형을 템플릿 인자로 지정하면 `vector<Student>`를 만들 수 있다.

```

1 vector<Student> students;
2
3 students.push_back(Student("Alice", 85));
4 students.push_back(Student("Bob", 92));

```

8.2. 객체와 `find_if`

`vector<Student>`의 각 원소는 `Student` 객체이므로 람다의 매개변수도 `Student` 객체를 받을 수 있어야 한다.

```

1 [](const Student& student)
2 {
3     return student.score >= 90;
4 }

```

`const Student&`는 객체를 복사하지 않고 참조로 받아 읽기만 한다. `find_if()`가 반환한 반복자가 객체를 가리키므로 멤버에는 화살표 연산자로 접근할 수 있다.

```

1 cout << it->name << endl;
2 cout << it->score << endl;

```

이는 다음 표현과 같은 의미이다.

```

1 cout << (*it).name << endl;
2 cout << (*it).score << endl;

```

8.3. 최종 코드

```

1 #include <algorithm>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 using namespace std;
7
8 class Student
9 {
10 public:
11     string name;
12     int score;
13
14     Student(string name, int score)
15     {
16         this->name = name;

```

```

17     this->score = score;
18 }
19 };
20
21 int main()
22 {
23     vector<Student> students;
24
25     students.push_back(Student("Alice", 85));
26     students.push_back(Student("Bob", 92));
27     students.push_back(Student("Charlie", 78));
28     students.push_back(Student("David", 95));
29
30     auto it = find_if(
31         students.begin(),
32         students.end(),
33         [](const Student& student)
34         {
35             return student.score >= 90;
36         }
37     );
38
39     if (it != students.end())
40     {
41         cout << "First student above 90:" << endl;
42         cout << "Name: " << it->name << endl;
43         cout << "Score: " << it->score << endl;
44     }
45
46     int count = count_if(
47         students.begin(),
48         students.end(),
49         [](const Student& student)
50         {
51             return student.score >= 90;
52         }
53     );
54
55     cout << "Count: " << count << endl;
56
57     return 0;
58 }

```

8.4. 실습

실습

Book 클래스를 만들고 title과 price를 멤버로 저장한다. 네 권의 책을 vector<Book>에 저장한 뒤, find_if()로 30000원 이상인 첫 번째 책을 찾고 count_if()로 25000원 이상인 책의 개수를 계산한다.

```

1 #include <algorithm>
2 #include <iostream>
3 #include <string>
4 #include <vector>
5
6 using namespace std;
7

```

```
8 class Book
9 {
10 public:
11     string title;
12     int price;
13
14     Book(string title, int price)
15     {
16         this->title = title;
17         this->price = price;
18     }
19 };
20
21 int main()
22 {
23     vector<Book> books;
24
25     books.push_back(Book("C++ Basics", 25000));
26     books.push_back(Book("Python Guide", 18000));
27     books.push_back(Book("Algorithms", 32000));
28     books.push_back(Book("Data Science", 29000));
29
30     auto it = find_if(
31         books.begin(),
32         books.end(),
33         [](const Book& book)
34         {
35             return book.price >= 30000;
36         }
37     );
38
39     int count = count_if(
40         books.begin(),
41         books.end(),
42         [](const Book& book)
43         {
44             return book.price >= 25000;
45         }
46     );
47
48     if (it != books.end())
49     {
50         cout << "First expensive book:" << endl;
51         cout << "Title: " << it->title << endl;
52         cout << "Price: " << it->price << endl;
53     }
54     else
55     {
56         cout << "There is no book above 30000" << endl;
57     }
58
59     cout << "Books above 25000: " << count << endl;
60
61     return 0;
62 }
```

9. 개념 연결

Day 12의 각 기능은 독립적으로 끝나는 것이 아니라 이전에 배운 C++ 개념과 연결된다.

이전 학습 내용	Day 12에서의 연결
참조	람다의 참조 캡처와 <code>const T&</code> 매개변수
클래스	<code>vector<Student></code> , <code>vector<Book></code>
템플릿	<code>vector<T></code> 와 STL 컨테이너의 기반
반복자	<code>find_if()</code> 의 반환값과 <code>begin()</code> , <code>end()</code>
람다	STL 알고리즘에 조건과 변환 규칙 전달
컨테이너 어댑터	<code>queue</code> 에서 <code>priority_queue</code> 로 확장

핵심 포인트

STL 알고리즘은 컨테이너의 원소를 직접 알고 있는 것이 아니라 반복자로 범위를 전달받는다. 람다는 그 범위의 원소를 어떻게 검사하거나 변환할지를 정의한다. 클래스 객체도 같은 방식으로 처리할 수 있으므로, 클래스와 STL은 실제 프로그램에서 자연스럽게 함께 사용된다.

10. 종합 정리

핵심 포인트

- `priority_queue`는 기본적으로 가장 큰 값을 `top()`에서 제공한다.
- 값 캡처는 바깥 변수의 값을 복사하고, 참조 캡처는 바깥 변수 자체를 참조한다.
- `find_if()`는 조건을 만족하는 첫 번째 원소의 반복자를 반환한다.
- `count_if()`는 조건을 만족하는 원소의 개수를 반환한다.
- `predicate`는 원소를 검사하여 `true` 또는 `false`를 반환하는 조건 함수이다.
- `transform()`은 각 원소에 변환 함수를 적용하고 결과를 출력 범위에 저장한다.
- 다른 벡터에 `transform()` 결과를 저장할 때는 저장 공간을 준비해야 한다.
- 클래스 객체도 STL 컨테이너에 저장할 수 있다.
- 객체를 읽기만 하는 람다 매개변수에는 `const T&`를 사용할 수 있다.
- 클래스, 컨테이너, 반복자, 람다, 알고리즘을 함께 사용하면 실제 데이터를 유연하게 처리할 수 있다.

11. 실습 파일 구성

실습

Day 12 파일은 다음과 같이 관리한다.

- 01_priority_queue.cpp
- 02_lambda_capture.cpp
- 03_find_if.cpp
- 04_count_if.cpp
- 05_transform.cpp
- 06_object_container.cpp
- practice/p01_priority_queue.cpp
- practice/p02_lambda_capture.cpp
- practice/p03_find_if.cpp
- practice/p04_count_if.cpp
- practice/p05_transform.cpp
- practice/p06_object_container.cpp